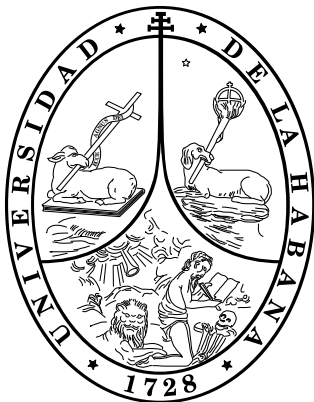




Universidad de La Habana
Facultad de Matemática y Computación

Informe Técnico de Software

Sistema web para el autorreporte de Eventos
Supuestamente Atribuibles a la Vacunación o
Inmunización (ESAVI) en el Instituto Finlay de
Vacunas (IFV)

Proyecto:	Sistema web para el autorreporte de ESAVI en el IFV
Organización:	Instituto Finlay de Vacunas (IFV)
Autor:	Jabel Resendiz Aguirre
Facultad:	Facultad de Matemática y Computación
Fecha:	19 de agosto de 2026
Clasificación:	Público

Índice

Control de Versiones	4
1. Resumen Ejecutivo	5
2. Alcance y Objetivos	5
2.1. Propósito del sistema	5
2.2. Alcance del proyecto	5
3. Requisitos del Sistema	6
3.1. Requisitos funcionales	6
3.2. Requisitos no funcionales	7
3.3. Requisitos de entorno	8
4. Contexto y Casos de Uso	9
4.1. Descripción del sistema	9
4.2. Identificación de los actores	9
4.3. Diagrama de casos de uso	9
4.4. Especificación de casos de uso principales	10
4.5. Reglas de negocio	12
5. Arquitectura del Sistema	13
5.1. Vista arquitectónica general	13
5.2. Componentes principales	14
5.3. Patrones de diseño aplicados	15
5.4. Comunicación entre componentes	16
6. Tecnologías y Dependencias	17
6.1. Stack tecnológico	17
6.2. Dependencias externas	17
6.3. Justificación técnica	18
7. Estructura del Código Fuente	18
7.1. Organización de directorios	18
7.2. Convenciones de código	19
7.3. Gestión de repositorio	19
8. Configuración y Entorno	20
8.1. Requisitos de hardware	20
8.2. Variables de entorno	20
8.3. Automatización de tareas	20
9. Modelo de Datos y Persistencia	21
9.1. Diagrama entidad-relación	21
9.2. Estrategia de respaldo	22

10.API e Integraciones	22
10.1. Diseño de API	22
10.2. Documentación de endpoints	23
10.3. Integraciones externas	23
11.Pruebas Funcionales	24
11.1. Estrategia de pruebas funcionales	24
11.2. Evidencias de ejecución	25
11.2.1. Flujo completo del manejo de reportes ESAVI	25
11.2.2. Gestión de acceso y registro de médico evaluador	27
11.3. Análisis de resultados	28
12.Rendimiento y Escalabilidad	29
12.1. Métricas de rendimiento	29
12.2. Estrategia de escalabilidad	29
12.3. Pruebas de carga realizadas	29
12.3.1. Resultados: Creación de reportes	30
12.3.2. Resultados: Búsqueda de reportes	30
12.3.3. Resultados: Escenario combinado	31
12.4. Resumen de resultados	32
13.Seguridad	32
13.1. Autenticación basada en JWT y Refresh Token	32
13.1.1. JSON Web Token (JWT)	32
13.1.2. Refresh Token en cookie HttpOnly	33
13.2. Autorización basada en roles (RBAC)	33
13.3. Protección de datos sensibles	34
13.3.1. Hashing de contraseñas	34
13.3.2. Cifrado de datos sensibles y índice ciego	34
13.4. Prevención de envíos duplicados (Idempotencia)	35
13.5. Protección ante ataques automatizados	35
13.5.1. CAPTCHA	35
13.5.2. Limitación de velocidad (Rate Limiting)	36
13.6. Gestión de secretos	36
14.Monitoreo y Logs	36
14.1. Estrategia de logging	36
14.2. Manejo centralizado de errores	37
14.3. Estado actual del monitoreo	37
14.4. Recomendaciones para el entorno productivo	37
15.Despliegue e Integración Continua	38
15.1. Estado actual del despliegue	38
15.2. Pipeline CI/CD	38
15.3. Integración con servicios externos	39
15.4. Gestión de infraestructura	39

15.5. Recomendaciones para producción	39
16. Gestión de Riesgos	40
16.1. Identificación de riesgos	40
16.2. Planes de mitigación	40
16.3. Análisis de riesgos mitigados	40
17. Accesibilidad y Experiencia de Usuario	41
17.1. Cumplimiento de accesibilidad	41
17.2. Principios de UX aplicados	41
17.3. Evaluación de usabilidad	42
17.4. Requisitos previos a la puesta en producción	43
18. Limitaciones y Trabajo Futuro	43
18.1. Limitaciones conocidas	43
18.2. Roadmap de mejoras	44
A. Manual de Instalación para Evaluadores	44
A.1. Requisitos previos	44
A.2. Pasos para el entorno de desarrollo con contenedores	44
A.3. Verificaciones después del arranque	45
A.4. Ejecución de la API sin Docker	46
A.5. Solución de problemas comunes	46
B. Glosario de Términos Técnicos	46
C. Referencias Bibliográficas y Normativas	47

Control de Versiones

Cuadro 1: Historial de cambios del documento

Versión	Fecha	Autor	Descripción del cambio
1.0.0	15/08/2026	Jabel Resendiz	Versión inicial del informe técnico

1. Resumen Ejecutivo

El proyecto consiste en un sistema web orientado al autorreporte de eventos supuestamente atribuibles a la vacunación o inmunización (ESAVI) para el Instituto Finlay de Vacunas. La solución permite a los reporteros registrar incidentes relacionados con la vacunación, hacer seguimiento del estado de cada notificación y brindar herramientas de revisión a profesionales médicos, responsables de sección y administradores. La arquitectura combina un frontend en React y TypeScript con un backend en ASP.NET Core 8, con persistencia en MySQL, mensajería con RabbitMQ y despliegue mediante contenedores Docker.

El sistema se encuentra en una etapa de desarrollo y validación funcional, con componentes operativos para autenticación, gestión de reportes, dashboards, catálogos de vacunas y síntomas, asignación médica, manejo de centros de vacunación y lotes. La solución ofrece un flujo de trabajo que va desde la captura pública del reporte hasta la revisión y administración interna, con control de acceso por roles y mecanismos como JWT, refresh tokens y limitación de peticiones.

Los beneficios esperados del sistema son la reducción de los tiempos de registro y seguimiento de los ESAVI, la estandarización de la información recopilada, la trazabilidad de cada caso y la mejora en la toma de decisiones por parte de las áreas técnicas y médicas del instituto.

2. Alcance y Objetivos

2.1. Propósito del sistema

El sistema responde a la necesidad de contar con un canal digital seguro y formalizado para el reporte de eventos adversos asociados a vacunas. Su propósito es centralizar la información de los casos reportados, facilitar su revisión por personal médico y permitir a la organización mantener trazabilidad y control sobre los incidentes notificados.

La plataforma también busca reducir la dependencia de procesos manuales, mejorar la calidad de la información y ofrecer una vista operativa para la administración de reportes, usuarios, catálogos y asignaciones internas.

2.2. Alcance del proyecto

Alcance incluido:

- Registro público de reportes de ESAVI desde el frontend.
- Seguimiento del estado de cada reporte mediante número de notificación.
- Autenticación de usuarios con roles de administrador, responsable de sección, revisor médico y usuario estándar.
- Dashboards y consultas para monitoreo de reportes y métricas generales.
- Gestión de catálogos de vacunas, síntomas, manufacturas, centros de vacunación y lotes.

- Asignación y reasignación de reportes para revisión médica.
- Detección y resolución de reportes duplicados.

Alcance excluido:

- Aplicación móvil nativa.
- Funcionamiento en modo offline.
- Integración directa con sistemas externos de farmacovigilancia o plataformas regulatorias.
- Soporte multilenguaje o adaptación completa a múltiples regiones.

Supuestos:

- El entorno de desarrollo cuenta con Docker, MySQL y RabbitMQ disponibles.
- Los usuarios acceden desde navegadores modernos con conexión a internet.
- Los datos de salud reportados requieren tratamiento confidencial y restringido a usuarios autorizados.

Restricciones:

- La implementación actual está enfocada a un entorno interno y no contempla despliegue distribuido a gran escala.
- El repositorio visible no incluye una suite automatizada de pruebas completa.
- La configuración de variables de entorno y secretos depende del operario de despliegue.

3. Requisitos del Sistema

3.1. Requisitos funcionales

Los requisitos funcionales definen las capacidades que el sistema debe ofrecer para la captura, gestión, seguimiento y análisis de los reportes de ESAVI, abarcando los diferentes perfiles de usuario identificados en el diagrama de casos de uso.

Cuadro 2: Requisitos funcionales

ID	Requisito	Prioridad
RF-01	Registrar reportes ESAVI mediante formulario web estructurado.	Crítica
RF-02	Generar código único de notificación por reporte exitoso.	Crítica
RF-03	Validar campos obligatorios antes del envío del reporte.	Crítica
RF-04	Almacenar reportes en base de datos centralizada.	Crítica
RF-05	Gestionar estado del reporte (enviado, revisión, aprobado, rechazado, cerrado).	Crítica
RF-06	Ofrecer consulta con búsqueda, filtrado y ordenación según rol.	Crítica
RF-07	Autenticar usuarios y administrar roles para acceso a funcionalidades.	Crítica
RF-08	Generar estadísticas y visualizaciones gráficas de eventos y rendimiento.	Alta
RF-09	Exportar información a PDF y Excel para análisis institucionales.	Alta
RF-10	Enviar notificaciones automáticas según eventos del flujo y rol del usuario.	Alta
RF-11	Detectar duplicados automáticamente y permitir veredicto manual.	Alta
RF-12	Reasignar reportes cuya fecha límite de evaluación haya sido superada.	Alta

3.2. Requisitos no funcionales

Los requisitos no funcionales definen las condiciones de calidad bajo las cuales debe operar la solución. En la Tabla 3 se resumen los atributos considerados más relevantes.

Cuadro 3: Requisitos no funcionales

ID	Requisito	Categoría
RNF-01	Interfaz intuitiva con elementos estructurados (listas, botones, casillas), minimizando texto libre.	Usabilidad
RNF-02	Cumplir pautas WCAG 2.1 nivel AA para personas con discapacidades.	Accesibilidad
RNF-03	Adaptarse a escritorio, tabletas y dispositivos móviles.	Diseño responsivo
RNF-04	Compatible con Chrome, Firefox, Edge y Safari.	Compatibilidad
RNF-05	Procesar envío de reporte en menos de 3 segundos (sin latencia de red externa).	Rendimiento
RNF-06	Diseño modular con separación clara de componentes para facilitar mantenimiento y escalabilidad.	Mantenibilidad
RNF-07	Garantizar confidencialidad, integridad y disponibilidad mediante autenticación, control de acceso y cifrado.	Seguridad
RNF-08	Registrar acciones de usuarios (creación, modificación, actualización) para auditoría y trazabilidad.	Auditoría

3.3. Requisitos de entorno

El proyecto fue diseñado para operar en tres niveles de entorno, garantizando la consistencia mediante contenerización con Docker y facilitando la migración progresiva hacia la infraestructura institucional del IFV.

Cuadro 4: Requisitos de entorno

Componente	Desarrollo	Demostración	Producción futura
Frontend	Vite/React local	Vercel	Servidor institucional con HTTPS
Backend	API .NET 8 local	Railway	Servidor dedicado del IFV
Base de datos	MySQL 8 en contenedor	MySQL en Railway	MySQL administrado internamente
Mensajería	RabbitMQ local	RabbitMQ en Railway	RabbitMQ o servicio interno
Red	Localhost y puertos expuestos	Red interna o VPN	HTTPS con reglas de firewall

4. Contexto y Casos de Uso

4.1. Descripción del sistema

El sistema se denomina plataforma web para el autorreporte de ESAVI del Instituto Finlay de Vacunas. Es una aplicación web de arquitectura cliente-servidor, con frontend SPA en React y backend REST en ASP.NET Core. La solución está pensada para operación interna y de uso institucional, con acceso a través de navegadores modernos en entornos web.

4.2. Identificación de los actores

De acuerdo con el modelado funcional, el sistema cuenta con cinco actores principales que interactúan con la plataforma de gestión de vacunación y ESAVI:

- **Usuario Autenticado:** Actor abstracto que representa la base de permisos comunes. De él heredan todos los roles que requieren autenticación en el sistema.
- **Jefe de Vacunación Municipal:** Hereda de Usuario. Responsable de la supervisión local, encargado de la gestión del personal médico y la distribución de la carga de trabajo.
- **Médico Evaluador:** Hereda de Usuario. Profesional sanitario encargado del análisis clínico de los eventos reportados.
- **Administrador:** Hereda de Usuario. Representa al personal del IFV que supervisa el proceso y monitorea el comportamiento de la asignación y revisión de reportes en cada territorio.
- **Reportante:** Actor independiente sin autenticación. Representa a la persona que notifica un evento adverso al sistema.

4.3. Diagrama de casos de uso

La figura 1 recoge los casos de uso del sistema. El diagrama organiza las funcionalidades según el actor que las ejecuta y refleja las relaciones de extensión entre casos de uso.

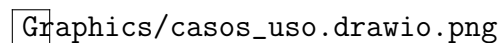
Graphics/casos_uso.drawio.png

Figura 1: Diagrama de casos de usos.

4.4. Especificación de casos de uso principales

A continuación se describe la lógica de negocio asociada a cada interacción:

- **Iniciar sesión:** Caso de uso base que permite el acceso controlado a la plataforma según las credenciales del usuario (correo electrónico y contraseña). Este caso de uso es común a los roles de jefe de vacunación municipal, médico evaluador y administrador, quienes heredan esta funcionalidad del actor usuario autenticado. El reportante, al ser un actor independiente sin autenticación, no tiene acceso a esta funcionalidad.

■ **Gestión de reportes (Reportante):**

- **Crear reporte ESAVI:** Permite el ingreso de un nuevo reporte mediante un formulario estructurado que captura información del sujeto vacunado, datos médicos previos, vacunación, eventos adversos y datos del reportante. Al completar el envío, el sistema genera un número de notificación único, envía un mensaje al jefe de vacunación del municipio correspondiente sobre el nuevo reporte. Este caso de uso posee una relación de tipo *extend* hacia **Exportar a PDF**, lo que indica que el usuario puede, opcionalmente, generar un comprobante documental tras completar el registro.
- **Consultar estado de reporte:** Permite al reportante verificar el estado de sus envíos previos ingresando el número de notificación.

■ **Supervisión Municipal (Jefe de Vacunación):**

- **Gestionar reportes:** Permite al jefe de vacunación visualizar los reportes de su municipio en una tabla filtrable por gravedad, vacuna, estado del reporte y ordenable por fecha. Desde esta vista, puede seleccionar un reporte y asignarlo a un médico evaluador disponible. Al asignar, el sistema registra la fecha límite de evaluación según la prioridad del reporte (ver RN-08). De manera opcional (*extend*), puede **Reasignar reporte:** cuando una asignación supera su plazo de evaluación sin que el médico la haya completado, el jefe reasigna el reporte a otro médico. La asignación anterior pasa a estado **cancelado** y crea una nueva con una fecha límite actualizada.
- **Gestionar duplicados:** Permite al jefe de vacunación acceder a una cola de revisión que lista los posibles duplicados detectados por el sistema (ver RN-09). Al seleccionar un caso, el sistema muestra una vista comparativa lado a lado entre el reporte original y la copia. El jefe emite un veredicto manual mediante una de estas acciones: **Confirmar duplicado** (descarta la copia del flujo y registra la resolución) o **Separar como caso nuevo** (libera la copia al flujo normal y registra la resolución). Este control de calidad es previo a la asignación y garantiza la integridad de los datos antes de que un caso llegue a un médico evaluador.
- **Visualizar panel de control municipal:** Ofrece un panel con indicadores agregados del municipio: reportes totales, pendientes, en revisión y completados; tiempos promedio de revisión y de asignación; promedio de asignaciones por reporte; distribución de reportes por rango de tiempo de completitud; distribución porcentual de estados por médico; carga activa de trabajo por médico; y los elementos más reportados: vacunas, síntomas y distribución por gravedad, junto con una línea de tiempo de reportes. Estos indicadores permiten al jefe identificar qué médicos están más ocupados, cuáles responden con mayor rapidez y tomar decisiones sobre la distribución de la carga de trabajo.
- **Gestionar médicos:** Permite al jefe de vacunación dar de alta a los médicos evaluadores bajo su jurisdicción. El jefe ingresa el correo electrónico y el número de registro profesional del médico. El sistema envía un enlace de activación al correo del médico, quien completa su registro estableciendo su contraseña y confirmando

su número de registro profesional. Este mecanismo garantiza que solo el médico legítimo active su cuenta y que la contraseña no sea conocida por terceros.

■ **Gestión Clínica (Médico Evaluador):**

- **Gestionar reportes asignados:** Vista principal donde el médico visualiza los reportes que le han sido asignados por el jefe de vacunación de su municipio. Cada reporte muestra el número de notificación, la fecha de creación y la gravedad. La tabla permite filtrar y ordenar por antigüedad, gravedad u otros criterios para priorizar la carga de trabajo.
- **Revisar reporte:** El médico accede al detalle del reporte, analiza la información clínica del sujeto vacunado, los datos de vacunación y los eventos adversos descritos. A partir de esta información, determina la causalidad del evento, emite un dictamen y completa la revisión. La regla RN-08 establece el plazo máximo para completar esta evaluación según la prioridad del reporte. Al finalizar, el sistema notifica al jefe de vacunación sobre la conclusión de la evaluación.

■ **Administración y Configuración (Administrador):**

- **Actualizar catálogos:** Permite al administrador mantener actualizados los catálogos del sistema: vacunas, síntomas, lotes y centros de vacunación.
- **Gestionar jefes municipales:** Permite al administrador dar de alta a los jefes de vacunación municipal. El sistema envía un enlace de activación al correo electrónico del jefe, quien completa su registro estableciendo su contraseña.
- **Consultar reportes:** Permite al administrador visualizar una tabla con todos los reportes del sistema, filtrable por vacuna, gravedad, estado del reporte y provincia. Cada fila muestra información preliminar del reporte —edad, sexo, provincia, vacunas, eventos adversos, gravedad, estado y médico asignado— y permite acceder al detalle completo del reporte. De manera opcional (*extend*), puede **Exportar a Excel** la tabla filtrada y **Exportar a PDF** el detalle de un reporte.
- **Visualizar panel de control:** Ofrece un panel con indicadores agregados y gráficos interactivos. Incluye tendencias de reportes por estado, distribución geográfica por provincias, reportes por gravedad y por causalidad, indicadores de rendimiento —número de médicos activos, promedio de reportes por médico, tiempos de revisión y de asignación— y los elementos más reportados: vacunas, lotes por vacuna y síntomas.

4.5. Reglas de negocio

Las reglas de negocio definen las restricciones y políticas que rigen la operación del sistema:

- **RN-01. Alcance territorial del jefe de vacunación:** Un jefe de vacunación solo puede gestionar reportes y asignar médicos evaluadores exclusivamente dentro del municipio al que está adscrito.

- **RN-02. Asignación exclusiva:** Un reporte solo puede estar asignado a un único médico evaluador a la vez. Cualquier nueva asignación —ya sea inicial o por reasignación— cancela la anterior.
- **RN-03. Unicidad de vacuna por fecha:** El sistema no permite que un reportante seleccione la misma vacuna del catálogo más de una vez en la misma fecha para un mismo sujeto vacunado.
- **RN-04. Consistencia cronológica:** Las fechas registradas en un reporte deben respetar las siguientes restricciones. La fecha de nacimiento del sujeto y del reportante antecede a todas las fechas de vacunación y de inicio de eventos adversos. Cada fecha de inicio de evento adverso es posterior o igual a todas las fechas de vacunación. Si un evento adverso registra fecha de fin, esta debe ser posterior o igual a su fecha de inicio. Cualquier fecha de fallecimiento debe ser posterior o igual al inicio de los eventos adversos y anterior o igual a la fecha de reporte y a la fecha actual.
- **RN-05. Integridad del reporte:** Todo reporte debe contener al menos un evento adverso y una vacuna asociada para ser enviado.
- **RN-06. Gravedad derivada:** El sistema clasifica automáticamente un evento adverso como serio si el reportante marcó al menos uno de los criterios de gravedad registrados en el reporte. En caso contrario, lo clasifica como no serio. El usuario no interviene directamente sobre este campo.
- **RN-07. Priorización de reportes por nivel de riesgo:** El sistema clasifica cada reporte según el evento adverso más grave que contenga. La prioridad responde al siguiente criterio:
 - **Prioridad alta:** el evento es serio (resultó en muerte, hospitalización, visita al doctor o a emergencias, discapacidad permanente o anomalía congénita).
 - **Prioridad media:** el evento no es serio pero la intensidad del síntoma es severa, o el sujeto vacunado continúa sin recuperarse o presenta secuelas.
 - **Prioridad baja:** el evento no cumple ninguna de las restricciones anteriores (evento no serio, sujeto recuperado, en recuperación, desconocido, intensidad breve).
- **RN-08. Plazo de evaluación según prioridad:** Los reportes de prioridad alta deben evaluarse en un máximo de 24 horas; los de prioridad media, en un máximo de 5 días; los de prioridad baja, en un máximo de 7 días. Cada asignación registra su fecha límite al momento de crearse. Superado el plazo, el sistema notifica al médico evaluador y al jefe de vacunación correspondiente, pero no modifica automáticamente el estado de la asignación. Para determinar si una revisión cumplió el plazo, el sistema compara la fecha en que finalizó con la fecha límite registrada.
- **RN-09. Detección de posibles duplicados:** Al registrarse un nuevo reporte, el sistema verificará si ya existe en la base de datos otro reporte que comparta el mismo identificador del sujeto vacunado, la misma fecha de vacunación y la misma vacuna. En caso de coincidencia, el sistema insertará automáticamente un registro en la tabla de

control de duplicados, estableciendo: el reporte preexistente como `original`, el reporte recién creado como `copia`, y el estado inicial como `posible_duplicado`. Un reporte que figure como `copia` en dicha tabla con estado `posible_duplicado` no podrá ser asignado a ningún evaluador ni avanzar en el flujo de farmacovigilancia hasta que un usuario especializado emita un veredicto manual que confirme o descarte la duplicidad.

5. Arquitectura del Sistema

5.1. Vista arquitectónica general

La arquitectura del sistema sigue un patrón por capas, con separación entre frontend, API, lógica de aplicación, dominio e infraestructura. El frontend actúa como cliente SPA que consume servicios REST del backend. El backend expone controladores REST, utiliza servicios de aplicación y un repositorio/unidad de trabajo para encapsular el acceso a datos. La infraestructura gestiona MySQL, RabbitMQ y los procesos de inicialización y logging.

La figura 2 ilustra la organización completa del sistema bajo el modelo de arquitectura limpia, mostrando cómo cada capa se relaciona con las adyacentes y cómo las dependencias apuntan siempre hacia el núcleo del dominio.

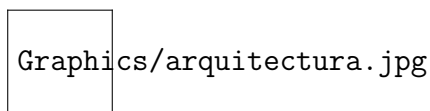


Figura 2: Vista arquitectónica general del sistema bajo el modelo de arquitectura limpia. Fuente: elaboración propia.

La arquitectura se organiza en cuatro niveles concéntricos que reflejan la regla de dependencia de la arquitectura limpia: las capas externas dependen de las internas, pero nunca al revés. En el centro se encuentra la capa de dominio, que contiene las entidades de negocio y las reglas fundamentales del sistema. Alrededor de esta, la capa de aplicación orquesta los casos de uso y define las interfaces que la infraestructura debe implementar. La capa de infraestructura proporciona las implementaciones concretas de persistencia, mensajería y servicios externos. Finalmente, la capa de presentación expone la funcionalidad a través de una API REST que es consumida por el frontend.

El flujo de comunicación entre componentes sigue un orden descendente que respeta esta organización:

- El **frontend** (React + TypeScript) realiza peticiones HTTP/JSON al backend mediante Axios.
- La **API REST** (ASP.NET Core 8) recibe estas peticiones y las traduce en invocaciones a los casos de uso de la capa de aplicación.
- La **capa de aplicación** aplica las validaciones necesarias, orquesta la lógica de negocio y utiliza los repositorios definidos mediante interfaces para acceder a los datos.

- La **capa de infraestructura** implementa estas interfaces proporcionando acceso a MySQL mediante EF Core, gestiona la mensajería a través de RabbitMQ y registra las operaciones mediante Serilog.
- El **dominio** contiene las entidades y objetos de valor que modelan el negocio, sin ningún tipo de dependencia externa.

5.2. Componentes principales

La tabla 5 desglosa los componentes fundamentales que conforman el sistema, especificando su responsabilidad, la tecnología empleada y sus dependencias dentro de la arquitectura.

Cuadro 5: Componentes principales

Componente	Responsabilidad	Tecnología	Dependencias
Frontend SPA	Renderiza vistas, formularios, dashboards y navegación por roles	React + TypeScript + Vite	Axios, React Router, MUI/-Radix
API REST	Expone endpoints para autenticación, reportes, catálogos y dashboards	ASP.NET Core 8	Application, Infrastructure
Capa de aplicación	Valida reglas de negocio y orquesta casos de uso	Services + DTOs + Validators	Domain, Infrastructure
Persistencia	Almacena entidades de reportes, usuarios, vacunas y revisiones	EF Core + MySQL	Unit of Work, Repositories
Mensajería	Soporta flujos asíncronos y procesamiento de eventos	RabbitMQ	Consumers, BackgroundServices

Cada componente ha sido diseñado con responsabilidades claramente delimitadas para facilitar el mantenimiento y la evolución del sistema. La separación entre la capa de aplicación y la de infraestructura permite que los cambios tecnológicos —como migrar de MySQL a PostgreSQL o sustituir RabbitMQ por Azure Service Bus— no afecten la lógica de negocio central, siempre que se respeten los contratos definidos en las interfaces.

5.3. Patrones de diseño aplicados

Para garantizar la calidad estructural del sistema y facilitar su evolución, se han aplicado varios patrones de diseño ampliamente reconocidos. La tabla 6 resume los patrones implementados, su categoría y su aplicación concreta en el sistema ESAVI.

Cuadro 6: Patrones aplicados

Patrón	Categoría	Aplicación en el sistema
Repositorio	Persistencia	Abstrae el acceso a datos mediante interfaces, permitiendo al dominio obtener y almacenar objetos sin conocer los detalles de persistencia. Declarado en aplicación e implementado en infraestructura con EF Core.
Unit of Work	Persistencia	Agrupar operaciones sobre repositorios y las persiste de forma atómica, garantizando que los cambios de una transacción se confirmen o descarten como una sola unidad .
Inyección de dependencias	Creacional	Servicios y repositorios registrados en el contenedor de ASP.NET Core, permitiendo que cada capa trabaje con interfaces sin conocer los detalles de construcción de sus dependencias .
Mapper (DTO)	Estructural	Convierte objetos entre capas mediante DTOs, transportando solo los datos necesarios en cada límite. Mantiene el dominio encapsulado y permite evolución independiente de capas.
Separación CQS	Arquitectónico	Separa operaciones en comandos (modifican estado) y consultas (retornan datos), facilitando pruebas unitarias y evolución independiente de cada interfaz.

La implementación del patrón Repositorio junto con Unit of Work garantiza que las operaciones de base de datos se realicen de forma atómica, manteniendo la consistencia de los datos incluso en operaciones que involucran múltiples entidades. La Inyección de dependencias actúa como mecanismo de desacoplamiento entre capas, permitiendo sustituir implementaciones sin afectar al núcleo del sistema. Por su parte, el Mapper mediante DTOs asegura que el dominio permanezca encapsulado y que cada capa pueda evolucionar de forma independiente, mientras que la Separación CQS facilita las pruebas unitarias y permite que cada interfaz evolucione según sus necesidades. Esta organización por capas, junto con los patrones descritos, conforma una arquitectura mantenible y flexible ante futuros cambios regulatorios o tecnológicos.

5.4. Comunicación entre componentes

La comunicación entre los componentes del sistema se divide en dos modalidades: síncrona y asíncrona. La comunicación síncrona entre frontend y backend se realiza mediante peticiones REST sobre HTTP/JSON, utilizando Axios en el cliente y controladores ASP.NET Core en el servidor. La comunicación asíncrona se apoya en RabbitMQ para procesos de mensajería o integración de eventos cuando el sistema requiera desacoplar tareas.

El flujo de comunicación sigue el siguiente patrón:

- El frontend envía peticiones HTTP al backend a través de la API REST.

- Los controladores de la API traducen estas peticiones en invocaciones a los casos de uso de la capa de aplicación.
- Los casos de uso coordinan las operaciones necesarias, interactuando con el dominio y utilizando los repositorios para acceder a los datos.
- Cuando se requiere procesamiento asíncrono —como el envío de notificaciones por correo electrónico o la generación de reportes en segundo plano—, la capa de aplicación publica mensajes en RabbitMQ.
- Los consumidores de RabbitMQ procesan estos mensajes de forma independiente, sin bloquear el flujo principal de la aplicación.

6. Tecnologías y Dependencias

6.1. Stack tecnológico

Cuadro 7: Stack tecnológico completo

Capa	Tecnología	Versión	Licencia
Frontend	React + TypeScript + Vite	18.3.1 / 7.x	MIT
Backend	ASP.NET Core 8 + C#	.NET 8.0	MIT
Base de datos	MySQL	8.0	GPL / comercial
Mensajería	RabbitMQ + MassTransit	3.x / 8.3.0	MPL-2.0 / Apache-2.0
Contenedores	Docker Compose	3.8 / Docker 24.x	Apache-2.0

6.2. Dependencias externas

Cuadro 8: Dependencias principales del proyecto

Paquete	Versión	Propósito	Licencia
react	18.3.1	Biblioteca principal del frontend	MIT
@mui/material	7.3.5	Componentes y diseño visual	MIT
react-router-dom	~7.17.0	Enrutamiento de vistas	MIT
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.0	Seguridad con JWT	MIT
Pomelo.EntityFrameworkCore.MySql	8.0.0	Provider de MySQL para EF Core	MIT
MassTransit.RabbitMQ	8.3.0	Integración con RabbitMQ	Apache-2.0
MailKit	4.15.1	Envío de correos electrónicos	MIT
QuestPDF	2026.6.0	Generación de documentos PDF	MIT

6.3. Justificación técnica

La solución se diseñó con un enfoque basado en una arquitectura parcialmente desacoplada, combinando un frontend moderno con un backend empresarial.

Para la capa de servicios, se eligió **ASP.NET Core 8** por su alta eficiencia de recursos y ecosistema integrado, que supera a Spring Boot en consumo de memoria y a NestJS en rendimiento en tareas intensivas. Para la persistencia, se optó por **MySQL** dentro del paradigma relacional, ofreciendo un equilibrio entre integración con Entity Framework Core, licencia de código abierto y despliegue multiplataforma frente a SQL Server y PostgreSQL. En mensajería, **RabbitMQ** permite desacoplar procesos asíncronos como notificaciones y alertas sin bloquear la experiencia del usuario. Para la interfaz, **React con TypeScript y Vite** proporciona el equilibrio entre eficiencia, flexibilidad y mantenibilidad necesario para interfaces con formularios dinámicos como los requeridos por el sistema de farmacovigilancia.

7. Estructura del Código Fuente

7.1. Organización de directorios

La estructura del proyecto refleja una separación clara entre frontend, backend y capas de negocio. La organización actual del repositorio es la siguiente:

```
esavi-report/  
  backend/  
    Finlay.PharmaVigilance.API/  
      Controllers/  
      Middleware/  
      Common/  
      Program.cs  
    Finlay.PharmaVigilance.Application/  
      Services/  
      DTOs/  
      Validators/  
      IServices/  
      IRepository/  
    Finlay.PharmaVigilance.Domain/  
      Entities/  
      Enum/  
      ValueObjects/  
      Events/  
    Finlay.PharmaVigilance.Infrastructure/  
      Repository/  
      Authentication/  
      Consumers/  
      EventBus/  
      Pdf/
```

```
Contact/
Migrations/
FinlayDbContext.cs
frontend/
  src/
    app/
    assets/
    services/
    styles/
    types/
  package.json
  vite.config.ts
  Dockerfile
```

Esta distribución responde a una arquitectura por capas: la API expone endpoints, la capa de aplicación implementa casos de uso, el dominio encapsula entidades y reglas de negocio, y la infraestructura gestiona acceso a datos, autenticación, mensajería y generación de documentos.

7.2. Convenciones de código

- En el backend, las clases y servicios se nombran en PascalCase; las variables y métodos siguen camelCase; las interfaces se identifican con prefijo I (por ejemplo, `IService`, `IRepository`).
- El dominio modela entidades, enums, value objects y eventos, manteniendo la lógica de negocio separada de la infraestructura.
- La infraestructura centraliza persistencia, autenticación JWT, mensajería RabbitMQ, envío de correos y generación de PDF.
- El frontend se organiza en módulos bajo `src/app`, servicios bajo `src/services` y recursos compartidos en `assets`, `styles` y `types`.
- La convención de desarrollo prioriza nombres descriptivos, responsabilidades únicas por clase y cohesión de funciones según el contexto del módulo.

7.3. Gestión de repositorio

- El repositorio se estructura en dos grandes bloques: `backend` y `frontend`, permitiendo independencia de despliegue y desarrollo paralelo.
- El backend usa una solución .NET con múltiples proyectos: API, Application, Domain e Infrastructure.
- El frontend está basado en React + Vite y gestiona la UI, rutas, integraciones con APIs y componentes reutilizables.

- La evolución del proyecto se apoya en commits organizados por funcionalidad, revisión de cambios mediante pull requests y control de versiones por ramas de trabajo.

8. Configuración y Entorno

8.1. Requisitos de hardware

Cuadro 9: Requisitos de hardware recomendados

Recurso	Desarrollo	Pruebas	Producción
CPU	4 núcleos	4-8 núcleos	8+ núcleos
RAM	8 GB	12-16 GB	16+ GB
Almacenamiento	25 GB SSD	50 GB SSD	100+ GB SSD
Red	100 Mbps	1 Gbps	1 Gbps redundante

8.2. Variables de entorno

El proyecto define variables de entorno para la configuración del backend y del frontend. En backend se configuran credenciales de base de datos, secreto JWT y conexión con RabbitMQ; en frontend se gestionan parámetros del entorno de ejecución y del consumo de servicios.

- `DB_PASSWORD`: contraseña del servicio de base de datos MySQL.
- `JWT_SECRET`: clave de firma para tokens de autenticación.
- `ConnectionStrings_`: cadenas de conexión usadas por Entity Framework y la API.
- `RABBITMQ_URL`: endpoint del broker para mensajería asíncrona.
- Variables del frontend para URLs de API y configuración del entorno de despliegue.

8.3. Automatización de tareas

El repositorio incluye scripts y comandos reutilizables para levantar el entorno y compilar cada capa del sistema:

- Backend: `dotnet build`, `docker compose up -d`, `./start_migrations.sh`.
- Frontend: `npm install`, `npm run dev`, `npm run build`.
- Docker Compose: orquesta MySQL y RabbitMQ para el entorno de desarrollo del backend.
- El flujo de trabajo permite levantar el sistema completo con dependencias y migraciones configuradas sin intervención manual significativa.

9. Modelo de Datos y Persistencia

9.1. Diagrama entidad-relación

El modelo de datos adopta un enfoque relacional, orientado a garantizar la integridad referencial y la trazabilidad de cada reporte desde su creación hasta su evaluación clínica. La figura 3 presenta el diagrama MERX (Modelo Entidad-Relación Extendido), el cual muestra las relaciones entre entidades, sus cardinalidades y las jerarquías de herencia para los roles de usuario.



Figura 3: Diagrama de modelo entidad-relación extendido (MERX). Fuente: elaboración propia.

El diseño se organiza en cinco módulos funcionales que segmentan las responsabilidades del sistema:

- **Módulo 1: Gestión de Usuarios y Roles** — Base de seguridad y control de acceso. Incluye jerarquía de herencia con las tablas *Usuario*, *Administrador*, *Jefe de Vacunación Municipal* y *Médico Evaluador*.
- **Módulo 2: Gestión de Reportes** — Núcleo del sistema. Integra la información del sujeto vacunado y del reportante mediante la jerarquía de herencia basada en la entidad *Person*.
- **Módulo 3: Vacunación** — Documenta los productos biológicos administrados, lotes, fabricantes y centros de salud.
- **Módulo 4: Evento Adverso** — Registra las manifestaciones clínicas (síntomas) detectadas tras la inmunización.
- **Módulo 5: Asignación y Evaluación** — Coordina el flujo de trabajo posterior a la notificación, gestionando alertas, asignaciones y dictámenes clínicos.

Todas las entidades del esquema emplean un identificador único universal (UUID) compuesto por 32 dígitos hexadecimales y 4 guiones que suman 36 caracteres, permitiendo generar el identificador sin depender de una secuencia centralizada de base de datos. Las tablas *Provincia* y *Municipio* constituyen la única excepción: emplean enteros autoincrementales por tratarse de catálogos geográficos con pocos registros y alta estabilidad.

Para acelerar las consultas, el diseño incorpora índices sobre campos de alta concurrencia como estados de reporte, fechas de creación, número de identidad (blind index) e identificadores de búsqueda frecuente.

Cuadro 10: Entidades principales del dominio

Entidad	Descripción
User	Usuarios del sistema con roles internos y credenciales para autenticación.
Person	Centraliza la identidad inmutable (nombre, género, número de identidad) de ciudadanos que pueden ser sujetos o reportantes.
Admin / SectionResponsible / MedicalEvaluator	Especializaciones de usuario con permisos y responsabilidades específicas.
Reporter	Información del reportante, incluyendo datos de contacto obligatorios.
AefiReport	Reporte principal del evento ESAVI, con número de notificación único.
VaccinatedSubject	Datos del sujeto vacunado (hereda de Person).
Vaccination	Información sobre la vacuna administrada, lote, fabricante y centro de vacunación.
Vaccine / Symptom / Lot / Manufacturer / VaccinationCenter	Catálogos de apoyo para la captura del reporte.
MedicalReviewAssignment	Registro de asignación de revisión médica de un reporte.
ReportDuplicate	Información sobre reportes duplicados o potencialmente relacionados.
Alert	Notificaciones automáticas generadas para jefes municipales sobre nuevos reportes.
Evaluation	Dictamen clínico final registrado por el médico evaluador, con escala de causalidad WHO-UMC.

9.2. Estrategia de respaldo

Actualmente, en el entorno de desarrollo, los respaldos se gestionan de forma manual mediante la herramienta `mysqldump`, generando copias completas de la base de datos bajo demanda. Esta estrategia es suficiente para la fase actual de desarrollo y pruebas, donde el volumen de datos es reducido y los riesgos de pérdida son bajos. Para la puesta en producción, se establecerá un proceso automatizado que incluya respaldos programados diarios, retención por un período mínimo de 7 días y almacenamiento en ubicación externa a la base de datos productiva.

10. API e Integraciones

10.1. Diseño de API

El backend expone una API REST documentada con Swagger. Los controladores están agrupados por dominio, como Authentication, Report, Catalog, VaccinationCenter, Medi-

calReview y Dashboard. El frontend consume estos endpoints a través de un cliente Axios centralizado, que además maneja refresh de tokens y tratamiento de errores.

10.2. Documentación de endpoints

Ejemplo de endpoints del sistema:

```
POST /api/Authentication/login
Descripción: Autentica un usuario y devuelve tokens de acceso y
refresh.

POST /api/Report/createPublic
Descripción: Crea un reporte ESAVI de forma pública desde el
frontend.

GET /api/Report/get-report
Descripción: Recupera el detalle de un reporte a partir de un
identificador.

GET /api/GetCatalog/vaccines/actives
Descripción: Devuelve las vacunas activas para el formulario de
reporte.

GET /api/VaccinationCenter/getByMunicipality
Descripción: Recupera los centros de vacunación asociados a una
provincia y municipio.
```

10.3. Integraciones externas

Cuadro 11: Integraciones externas

Servicio	Tipo	Datos intercambiados
FriendlyCaptcha	Captcha en formularios	Validación de humanidad en formularios públicos
EmailJS	Envío de correos electrónicos	Notificaciones a usuarios y reportantes sobre estados de reportes
MySQL	Base de datos relacional	Persistencia de reportes, usuarios y catálogos
RabbitMQ	Mensajería	Integración asíncrona y procesamiento de eventos

11. Pruebas Funcionales

11.1. Estrategia de pruebas funcionales

La validación de las funcionalidades críticas del sistema ESAVI se realizó mediante pruebas de caja negra, simulando operaciones de usuarios normales desde el navegador con datos sintéticos para verificar la persistencia y la respuesta del sistema ante entradas representativas.

La tabla 12 resume el estado de cumplimiento de las funciones principales: todos los casos de prueba seleccionados resultaron exitosos, confirmando la operatividad de la plataforma.

Cuadro 12: Resumen de casos de prueba funcionales

ID	Caso de prueba	Tipo	Resultado
CP01	Inicio de sesión con credenciales válidas	Positivo	✓
CP02	Registro de reporte ESAVI con datos completos	Positivo	✓
CP03	Consulta por número de notificación existente	Positivo	✓
CP04	Gestión de usuarios (médico o jefe de sección)	Positivo	✓
CP05	Asignación de reporte a médico	Positivo	✓
CP06	Resolución de reportes duplicados	Positivo	✓
CP07	Visualización del dashboard administrativo	Positivo	✓
CP08	Filtrado de reportes	Positivo	✓
CP09	Revisión médica del reporte	Positivo	✓
CP10	Inicio de sesión con credenciales inválidas	Negativo	✓
CP11	Registro de reporte con datos obligatorios ausentes	Negativo	✓
CP12	Consulta de número de notificación inexistente	Negativo	✓

11.2. Evidencias de ejecución

Esta sección presenta capturas de pantalla que evidencian la ejecución correcta de los principales flujos del sistema, validando la interacción entre el usuario y la interfaz.

11.2.1. Flujo completo del manejo de reportes ESAVI

Este flujo describe el ciclo completo de gestión de un reporte ESAVI, desde su creación por el usuario hasta su evaluación final por el médico evaluador y la consulta del estado actualizado del caso, tal como aparece en la figura [4](#).

(a) Creación de reporte ESAVI

(b) Confirmación de registro del reporte

(c) Asignación del reporte por el jefe de vacunación municipal a un médico evaluador

(d) Visualización del reporte por el médico evaluador asignado

(e) Actualización del estado del reporte tras la evaluación médica

Figura 4: Flujo del sistema ESAVI para la creación, asignación y evaluación de reportes de eventos adversos posteriores a la vacunación

11.2.2. Gestión de acceso y registro de médico evaluador

Este flujo, ilustrado en la figura 5, abarca el registro de un médico evaluador por parte del jefe municipal, el envío de una invitación por correo electrónico, el establecimiento de contraseña y el posterior inicio de sesión en el sistema, garantizando un acceso controlado al módulo de revisión de reportes.

Gestionar Médicos Revisores

Total de médicos revisores en el municipio: 16

Agregar Nuevo Médico Revisor

Nombre de Usuario * Juan Perez	Email * [redacted]00@gmail.com
Teléfono * 55 [redacted]	
Institución * Consultorio #23	Especialidad * Medicina General
Número de Registro Profesional * MED-19821-7890	Número de Identidad * 01012345678

[Registrar Médico](#) [Cancelar](#)

(a) Registro de médico evaluador por el jefe municipal

Invitación a la Plataforma de Farmacovigilancia

Se ha creado una cuenta para usted dentro del sistema de Farmacovigilancia del Instituto Finlay de Vacunas.

Para activar su cuenta y establecer sus credenciales de acceso, debe completar el proceso de verificación.

Durante la activación se le solicitará su **número de registro médico** como medida de validación de identidad.

[Activar cuenta](#)

Este enlace expirará en **24 horas** por motivos de seguridad. Si usted no esperaba esta invitación, puede ignorar este correo.

Atentamente,
Sistema de Farmacovigilancia — Instituto Finlay de Vacunas

Este correo fue enviado a [redacted]00@gmail.com

Email sent via [EmailJS.com](#)

(b) Correo de invitación con enlace para establecer contraseña

Completa tu registro

Ingresa tu número de registro profesional y tu nueva contraseña para activar tu cuenta.

Email de activación
[redacted]00@gmail.com

Número de registro profesional
MED-19821-7890

Nueva contraseña
.....

Confirmar contraseña
.....

[Completar registro](#)

[Volver a iniciar sesión](#)

(c) Página de establecimiento de contraseña

Sistema de Farmacovigilancia

Instituto Finlay de Vacunas

Plataforma de monitoreo y reporte de eventos adversos asociados a vacunas. Contribuya a la seguridad y eficacia de las inmunizaciones en Cuba.

[Ver Reportes Asignados](#)

JuanPerez (Profesional Evaluador) [Salir](#)

(d) Pantalla de inicio tras autenticación exitosa

Figura 5: Flujo de registro y activación de un médico evaluador en el sistema ESAVI

11.3. Análisis de resultados

Los 12 casos de prueba funcionales fueron ejecutados con éxito, sin desviaciones entre los resultados esperados y los obtenidos. Los flujos principales —creación, asignación, revisión y consulta de reportes— operaron de extremo a extremo conforme a los requerimientos.

El registro y activación de médicos evaluadores confirmó el control de acceso al sistema, y los casos negativos validaron que el sistema rechaza correctamente entradas inválidas. En conjunto, las pruebas funcionales demuestran que las funcionalidades críticas del sistema operan correctamente.

12. Rendimiento y Escalabilidad

12.1. Métricas de rendimiento

Las pruebas de rendimiento se ejecutaron con k6, evaluando las operaciones críticas del sistema en diferentes escenarios de carga. La tabla 13 define los objetivos de rendimiento establecidos para el sistema en condiciones normales de operación.

Cuadro 13: Objetivos de rendimiento (SLOs)

Métrica	Objetivo	Alerta	Crítico
Latencia p95	¡1000 ms	2000 ms	3000 ms
Throughput	20 req/s	15 req/s	10 req/s
Tasa de éxito	99.9 %	99 %	95 %
Solicitudes ¡umbral	100 %	95 %	90 %

12.2. Estrategia de escalabilidad

El sistema fue diseñado con una arquitectura de monolito modular que permite las siguientes estrategias de escalabilidad:

- **Vertical:** incremento de recursos (CPU, RAM) en el servidor que aloja el backend y la base de datos.
- **Horizontal:** replicación del backend mediante balanceo de carga, añadiendo instancias adicionales del servicio.
- **Caché:** implementación de caché distribuida (Redis) para reducir la carga en la base de datos.
- **Optimización de consultas:** uso de índices en columnas de alta concurrencia como `notification_number`.

12.3. Pruebas de carga realizadas

Las pruebas de carga emplearon k6 como herramienta de evaluación. Los escenarios ejecutados fueron:

- **Prueba de estrés incremental:** aumento progresivo de usuarios virtuales (VUs) para identificar el punto de saturación del sistema.

- **Prueba de volumen:** evaluación del impacto del crecimiento de la base de datos (1,000 a 100,000 reportes) en el rendimiento de búsqueda.
- **Escenario combinado:** simulación de carga realista con múltiples perfiles de usuario (ciudadanos, jefes municipales y médicos) operando simultáneamente.

12.3.1. Resultados: Creación de reportes

La prueba de estrés sobre la creación de reportes incrementó la carga desde 5 hasta 200 usuarios concurrentes. La tabla 14 resume los resultados obtenidos.

Cuadro 14: Resultados de creación de reportes

VUs	Latencia p95 (ms)	Latencia promedio (ms)	% ¡3000 ms
5	172.8	102.7	100 %
10	315.8	182.5	100 %
30	623.7	298.5	100 %
50	1158.1	530.8	100 %
70	2147.0	952.0	100 %
80	2851.3	1810.4	95.8 %
100	4033.2	2228.2	74.8 %
150	6706.2	4703.1	19.1 %
200	8005.9	5393.7	10.7 %

- **Zona óptima:** Hasta 70 VUs, la latencia p95 se mantiene por debajo de 3000 ms.
- **Zona de degradación:** A partir de 80 VUs, la latencia comienza a superar los 2000 ms.
- **Throughput:** Se estabilizó entre 20 y 26 req/s en todos los niveles de carga.
- **Tasa de éxito:** 100 % de solicitudes exitosas (status 201) en todos los niveles.

12.3.2. Resultados: Búsqueda de reportes

La prueba de búsqueda por número de notificación evaluó el impacto del volumen de datos en la latencia, variando el tamaño de la base de datos desde 1,000 hasta 100,000 reportes.

Cuadro 15: Resultados de búsqueda por notificación (p95 en ms)

VUs	10^3	4×10^3	10^4	5×10^4	10^5
5	48.4	79.6	55.4	85.3	110.5
10	94.6	108.1	111.0	145.2	185.8
30	320.2	358.1	354.5	430.7	520.3
50	553.8	624.6	723.9	780.4	950.6
70	787.9	830.2	862.2	1050.8	1350.2
100	1073.4	1200.0	1540.4	1750.3	2200.7
150	1637.2	1777.9	2027.3	2600.5	3100.4
200	2425.2	2377.3	2951.4	3250.0	3850.0

- Hasta 70 VUs, todas las curvas permanecen agrupadas por debajo de 1500 ms.
- Con 10^3 , 4×10^3 y 10^4 reportes, el umbral de 3000 ms no se supera incluso con 200 VUs.
- Con 5×10^4 y 10^5 reportes, se supera el umbral a partir de 200 VUs.
- Tasa de éxito: 100 % de solicitudes exitosas (status 202) en todos los niveles.

12.3.3. Resultados: Escenario combinado

El escenario combinado simuló una carga realista con 100 VUs distribuidos en tres perfiles: 75 ciudadanos, 10 jefes municipales y 15 médicos evaluadores.

Cuadro 16: Métricas globales del escenario combinado

Métrica	Valor
Solicitudes totales	2,382
Throughput global	36.8 req/s
Latencia promedio global	646.2 ms
Mediana global	471.8 ms
p(95) global	1623.4 ms
Tasa de éxito	99.5 %

Cuadro 17: Métricas por operación en escenario combinado

Operación	Solicitudes	Mediana (ms)	p95 (ms)	Outliers
Crear reporte	1078	858.28	2468.39	80
Buscar reporte	581	235.99	551.48	25
Pendientes	84	364.01	1143.59	8
Médicos	84	222.53	637.15	4
Asignar	84	433.23	1805.32	12
Asignaciones	365	278.22	783.83	18
Evaluar	68	382.17	911.90	4
Dashboard	38	368.06	1209.34	4

12.4. Resumen de resultados

Cuadro 18: Resumen de resultados de rendimiento

Operación	Capacidad máxima	p95 en carga normal
Creación de reporte	70 VUs sin degradación	1158 ms (50 VUs)
Búsqueda por notificación	200 VUs con 10^4 reportes	2951 ms (200 VUs, 10^4)
Escenario combinado	100 VUs	1623 ms (global)

El sistema demuestra un rendimiento robusto bajo carga concurrente. Las pruebas de estrés establecen que el sistema soporta hasta 70 usuarios concurrentes en operaciones de escritura sin degradación significativa, y más de 100 usuarios en escenarios mixtos. El throughput se estabiliza en aproximadamente 25 req/s para operaciones de escritura y 37 req/s en escenarios combinados. La búsqueda por número de notificación es la operación mejor optimizada, gracias al índice B-Tree implementado en la base de datos.

13. Seguridad

13.1. Autenticación basada en JWT y Refresh Token

El sistema emplea JSON Web Tokens (JWT) como mecanismo de autenticación sin estado, combinado con tokens de actualización (refresh tokens) almacenados en cookies HttpOnly. Esta combinación responde a dos objetivos contrapuestos: el token de acceso tiene una vida útil corta para limitar el daño en caso de robo, pero el usuario no necesita iniciar sesión repetidamente.

13.1.1. JSON Web Token (JWT)

Un JWT es un estándar abierto definido en el RFC 7519 que permite transmitir información verificable entre dos partes mediante un token firmado digitalmente [?]. El token consta de tres secciones codificadas en Base64 y separadas por puntos: encabezado, carga útil y firma. La carga útil contiene los *claims* —afirmaciones sobre el usuario, como su identificador, nombre y rol—, mientras que la firma garantiza que el token no fue alterado desde su emisión.

El sistema genera el JWT mediante HMAC-SHA256 como algoritmo de firma con una clave secreta simétrica almacenada en variables de entorno. Esta clave nunca aparece en el código fuente ni llega al cliente. El token incluye cuatro claims: identificador del usuario, nombre de usuario, identificador único del token y rol del usuario. La expiración queda configurada en 15 minutos.

El middleware de ASP.NET Core valida cada solicitud entrante verificando que el emisor y la audiencia coincidan con los valores configurados, que la firma sea correcta y que el token no haya expirado.

13.1.2. Refresh Token en cookie HttpOnly

Para compensar la corta vida del JWT sin sacrificar seguridad, el sistema implementa refresh tokens. Un refresh token es un valor aleatorio generado criptográficamente mediante `RandomNumberGenerator` de .NET, que produce 64 bytes de entropía y los codifica en Base64. Este valor reside en la base de datos junto con su fecha de vencimiento (8 horas) y el identificador del usuario.

El refresh token se transmite en una cookie con las siguientes propiedades de seguridad:

Cuadro 19: Configuración de seguridad de la cookie del refresh token

Propiedad	Propósito
<code>HttpOnly = true</code>	Impide que JavaScript acceda al token, neutralizando ataques XSS
<code>Secure = true</code>	Restringe la transmisión a conexiones HTTPS
<code>SameSite = None</code>	Permite envío desde dominios distintos (frontend/backend separados)
<code>Expires = 8h</code>	Caducidad del refresh token

Además, cada vez que el cliente utiliza un refresh token para obtener un nuevo JWT, el sistema aplica rotación: revoca el token anterior y emite uno nuevo. Si un atacante logra robar un refresh token, la rotación limita su utilidad a una única renovación.

13.2. Autorización basada en roles (RBAC)

Una vez que el middleware de autenticación valida el JWT y establece la identidad del usuario, el sistema delega el control de acceso en el atributo `[Authorize]` para restringir rutas por rol. El atributo `[Authorize(Roles = "SectionResponsible")]` indica al middleware de autorización que solo los usuarios cuyo claim `Role` coincida con ese valor pueden ejecutar el método.

La tabla 20 define la matriz de permisos según el rol del usuario:

Cuadro 20: Matriz de permisos (RBAC)

Recurso/Acción	Admin	SectionResponsible	MedicalEvaluator	Guest
Usuarios - Gestionar	Sí	No	No	No
Reportes - Crear	Sí	Sí	No	Sí
Reportes - Ver todos	Sí	Sí	No	No
Reportes - Ver asignados	Sí	Sí	Sí	No
Reportes - Editar	Sí	Sí	Sí	No
Asignar médicos	Sí	Sí	No	No
Evaluar reportes	Sí	No	Sí	No
Configuración del sistema	Sí	No	No	No
Consultar estado de reporte	Sí	Sí	Sí	Sí

Para que la capa de aplicación pueda acceder a la identidad del usuario autenticado sin que el frontend la envíe manualmente, el sistema implementa el servicio `UserContextService`. Este servicio extrae el claim `NameIdentifier` del token JWT validado —disponible en el `HttpContext` de la solicitud— y lo expone mediante el método `GetUserId()`. Este enfoque elimina la necesidad de que cada ruta reciba explícitamente el identificador del usuario como parámetro.

13.3. Protección de datos sensibles

13.3.1. Hashing de contraseñas

La documentación oficial de Microsoft describe que el `PasswordHasher` de Identity implementa el algoritmo PBKDF2 (Password-Based Key Derivation Function 2), estándar del RFC 2898 que deriva una clave segura a partir de una contraseña [1]. El algoritmo toma la contraseña del usuario, la combina con un *salt* criptográficamente aleatorio de 128 bits y aplica repetidamente la función HMAC-SHA256 durante 600,000 iteraciones como valor configurado [2].

El formato del hash generado produce una cadena con la siguiente estructura:

`Base64(salt).Base64(subkey)`

Cuadro 21: Propiedades de seguridad del mecanismo de hashing

Propiedad	Descripción
Unicidad del hash	Dos usuarios con la misma contraseña producen hashes completamente diferentes por el <i>salt</i> único
Irreversibilidad computacional	PBKDF2 es una función unidireccional; el elevado número de iteraciones ralentiza ataques por fuerza bruta
Invalidación de sesiones previas	La propiedad <code>SecurityStamp</code> invalida tokens anteriores al cambiar la contraseña

13.3.2. Cifrado de datos sensibles y índice ciego

Para armonizar la protección de datos sensibles con la necesidad de detectar reportes duplicados, la infraestructura implementa un índice ciego sobre el número de identidad del ciudadano.

La entidad persona almacena dos campos derivados del número de identidad:

- `IdentityNumberEncrypted`: preserva el valor original mediante cifrado AES-256 para su recuperación clínica.
- `IdentityNumberBlindIndex`: contiene un hash HMAC-SHA256 generado con una clave criptográfica distinta de la usada para el cifrado. Esta columna porta un índice que permite al motor de base de datos localizar el registro en tiempo logarítmico.

Cuadro 22: Medidas de protección de datos

Medida	Estándar	Aplicación en el sistema
En tránsito	TLS 1.2+	HSTS, certificados válidos, conexiones HTTPS forzadas
En reposo	AES-256	Cifrado a nivel de campo para números de identidad
Hashing de contraseñas	PBKDF2-SHA256	600,000 iteraciones, salt de 128 bits
Datos sensibles en logs	Enmascaramiento	Números de identidad y correos no se registran en texto plano

13.4. Prevención de envíos duplicados (Idempotencia)

Para evitar duplicados por dobles clics, reintentos automáticos por fallos de red o pérdidas de conectividad, la aplicación cliente implementa un mecanismo de idempotencia basado en una clave única asociada a cada operación de envío.

La clave surge de `crypto.randomUUID()`, función estandarizada que produce identificadores UUID v4 mediante un generador criptográficamente seguro. Una vez creada, la clave permanece almacenada en memoria del navegador hasta que el servidor responde satisfactoriamente.

El backend almacena la clave en una columna indexada con restricción de unicidad en la base de datos, lo que garantiza la creación de un único reporte incluso bajo escenarios de concurrencia extrema [3].

13.5. Protección ante ataques automatizados

13.5.1. CAPTCHA

Las rutas públicas del sistema —inicio de sesión, registro de reportes, consulta por número de notificación— están expuestas a ataques automatizados. Como primera barrera, el sistema exige que cada solicitud incluya un token de Friendly CAPTCHA, un servicio basado en prueba de trabajo (proof-of-work) que resuelve el desafío criptográfico en segundo plano sin interrumpir al usuario [4].

Para integrar este servicio, es necesario crear una cuenta en el sitio oficial de Friendly CAPTCHA y registrar una nueva aplicación. El proceso genera dos claves:

- **Sitekey**: identificador público que se incrusta en el frontend para inicializar el widget.
- **Secret key**: clave privada que se almacena exclusivamente en el backend para verificar los tokens recibidos.

Esta separación garantiza que el proceso de verificación solo pueda realizarse desde el backend, impidiendo que un atacante pueda falsificar tokens válidos [4].

13.5.2. Limitación de velocidad (Rate Limiting)

La limitación de velocidad restringe el número de solicitudes que un cliente puede realizar a la API en un período de tiempo determinado. Su propósito es proteger el sistema contra ataques de fuerza bruta, enumeración de recursos y denegación de servicio [5].

Cuadro 23: Políticas de rate limiting configuradas

Política	Límite	Ventana	Aplicación
Auth	3 solicitudes	1 minuto	Inicio de sesión
Command	5 solicitudes	1 minuto	Creación de reportes
ReportDaily	30 solicitudes	24 horas	Creación de reportes por IP
CommandDaily	20 solicitudes	24 horas	Operaciones administrativas
Query	60 solicitudes	1 minuto	Consultas y lecturas

13.6. Gestión de secretos

Las siguientes buenas prácticas se aplican para la gestión de secretos en el sistema:

- **Nunca hardcodear secretos:** Las claves JWT, contraseñas de base de datos y credenciales de servicios externos se almacenan en variables de entorno.
- **Separación por entorno:** Existen conjuntos de variables diferentes para desarrollo, pruebas y producción.
- **Rotación de claves:** Las claves JWT pueden rotarse sin afectar a los usuarios activos.
- **Acceso restringido:** Solo personal autorizado tiene acceso a las variables de entorno de producción.

En producción, se recomienda adoptar soluciones más avanzadas como HashiCorp Vault o AWS Secrets Manager para la gestión centralizada de secretos.

14. Monitoreo y Logs

14.1. Estrategia de logging

La capa de backend de ESAVI Report implementa un sistema de logging basado en Serilog, con salida simultánea a consola y archivos de log diarios. La configuración se realiza en el archivo `Program.cs` mediante `builder.Host.UseSerilog();`, junto con `WriteTo.Console()` y `WriteTo.File("logs/app-.log", rollingInterval: RollingInterval.Day)`. Esto permite mantener trazabilidad de eventos críticos, errores de validación, ejecuciones de servicios y fallas operativas durante la vida del sistema.

Los niveles configurados son:

- **INFO:** Operaciones normales y flujos exitosos.

- **WARN:** Eventos inusuales que no impiden la operación.
- **ERROR:** Fallos recuperables que afectan una operación específica.
- **CRITICAL:** Fallos graves que requieren atención inmediata.

Cada entrada registra datos estructurados como timestamp, nivel, servicio, identificador de correlación, mensaje y metadatos de contexto, lo que facilita la búsqueda y el diagnóstico en entornos con múltiples peticiones concurrentes.

14.2. Manejo centralizado de errores

El sistema implementa un middleware de manejo de errores que captura excepciones no controladas y responde con un formato JSON uniforme, reduciendo la ambigüedad en auditoría y soporte. Este middleware registra automáticamente cada excepción en Serilog con el nivel apropiado y el contexto completo de la petición.

14.3. Estado actual del monitoreo

Actualmente, el sistema cuenta con las siguientes capacidades de observabilidad:

- **Logs estructurados:** Implementados con Serilog en el backend.
- **Almacenamiento de logs:** Archivos rotativos diarios en el sistema de archivos.
- **Manejo de errores:** Middleware centralizado para excepciones no controladas.
- **Health checks:** *Pendiente de implementación para la versión productiva.*
- **Métricas y alertas:** *Pendiente de implementación.*

14.4. Recomendaciones para el entorno productivo

Para la puesta en producción, se recomienda implementar las siguientes mejoras:

Cuadro 24: Alertas propuestas para el entorno productivo

Métrica	Umbral	Severidad	Acción
CPU	$\geq 80\%$ (5 min sostenido)	Warning	Revisar procesos y escalar recursos
Errores 5xx	$> 1\%$ (2 min sostenido)	Critical	Revisión inmediata del backend y logs
Latencia p95	> 500 ms (10 min sostenido)	Warning	Optimizar consultas o escalar horizontalmente
Memoria RAM	$> 85\%$ (5 min sostenido)	Warning	Analizar fugas de memoria y reiniciar servicio
Disponibilidad	$< 99\%$ (5 min sostenido)	Critical	Alertar al equipo de operaciones

Para la observabilidad en producción, se sugiere adoptar progresivamente herramientas como:

- **Logs centralizados:** ELK Stack o Grafana Loki para agregar y consultar logs.
- **Métricas:** Prometheus + Grafana para almacenar y visualizar métricas.
- **APM:** New Relic o Datadog (opcional, según disponibilidad).

El siguiente paso de madurez operativa es centralizar los logs en un sistema de agregación y configurar dashboards que permitan correlacionar incidentes entre frontend, API y base de datos.

15. Despliegue e Integración Continua

15.1. Estado actual del despliegue

El sistema se despliega mediante contenedores Docker orquestados con Docker Compose. La arquitectura se compone de cuatro servicios principales:

Cuadro 25: Servicios del sistema

Servicio	Tecnología	Puerto
API Backend	ASP.NET Core 8	5137
Base de datos	MySQL 8.0	3306
Mensajería	RabbitMQ 3-management	5672 (AMQP), 15672 (UI)
Frontend	React + Vite + Nginx	8080

El backend se conecta a MySQL y RabbitMQ mediante variables de entorno definidas en `docker-compose.yml`. El frontend se sirve como una aplicación estática con Nginx.

15.2. Pipeline CI/CD

Actualmente, el repositorio no cuenta con pipelines de CI/CD automatizados. El despliegue se realiza de forma local mediante Docker Compose. Para producción, se recomienda incorporar un pipeline con las siguientes etapas:

- **Build:** Compilación de frontend y backend.
- **Pruebas:** Ejecución de pruebas unitarias y de integración.
- **Análisis de seguridad:** Escaneo de vulnerabilidades en dependencias.
- **Despliegue:** Publicación en entornos de staging y producción.

Herramientas recomendadas: GitHub Actions, GitLab CI o Jenkins.

15.3. Integración con servicios externos

Cuadro 26: Servicios externos integrados

Servicio	Propósito	Estado
EmailJS	Envío de notificaciones por correo	Prototipo (reemplazar en producción)
OpenWA / WhatsApp	Envío de mensajes por WhatsApp	Prototipo (reemplazar en producción)
FriendlyCaptcha	Validación de humanidad en formularios	Integrado

Los servicios de notificación (EmailJS y OpenWA) operan como servicios externos a la infraestructura Docker. En producción, deben reemplazarse por soluciones institucionales (SMTP propio y WhatsApp Business API).

15.4. Gestión de infraestructura

La infraestructura se gestiona con un enfoque basado en contenedores y configuración declarativa:

Cuadro 27: Gestión de infraestructura

Componente	Tecnología
Orquestación	Docker Compose
Contenerización	Docker
Variables de entorno	Archivos <code>.env</code>
Migraciones	Entity Framework Core + <code>dotnet-ef</code>
Logs	Serilog (archivos diarios)

Para un despliegue productivo, se recomienda evolucionar hacia orquestadores más robustos (Docker Swarm o Kubernetes), externalizar la configuración sensible, aplicar health checks y automatizar la construcción de imágenes.

15.5. Recomendaciones para producción

1. Migrar a servidores del Instituto Finlay (reemplazar Railway y Vercel).
2. Configurar SMTP institucional y WhatsApp Business API.
3. Implementar pipeline CI/CD automatizado.
4. Establecer health checks y monitoreo con Prometheus + Grafana.
5. Configurar respaldos automáticos de la base de datos.

16. Gestión de Riesgos

16.1. Identificación de riesgos

La siguiente tabla identifica los principales riesgos técnicos y operativos identificados durante el desarrollo del sistema, junto con su impacto y probabilidad estimada.

Cuadro 28: Identificación de riesgos

ID	Riesgo	Impacto	Probabilidad
R001	Dependencia de librerías mantenidas por una sola persona	Alto	Media
R002	Base de datos podría no escalar adecuadamente	Alto	Baja
R003	Fuga de datos por configuración incorrecta	Crítico	Media
R004	Dependencia de conocimiento tácito de un desarrollador clave	Alto	Alta
R005	Cambio inesperado en API de servicios externos (EmailJS, FriendlyCaptcha)	Alto	Media

16.2. Planes de mitigación

Para cada riesgo identificado, se han definido las siguientes estrategias de mitigación:

Cuadro 29: Planes de mitigación de riesgos

ID	Estrategia de mitigación	Estado
R001	Evaluar alternativas y mantener versiones estables documentadas	En curso
R002	Implementar índices en columnas críticas y planificar read replicas	Planificado
R003	Auditorías automáticas de configuración, variables de entorno y logs	Implementado
R004	Documentación obligatoria del código y arquitectura	En curso
R005	Implementar capa de abstracción para servicios externos	Implementado

16.3. Análisis de riesgos mitigados

Los riesgos R003 (fuga de datos) y R005 (cambios en APIs externas) cuentan con mitigaciones ya implementadas en el sistema: el cifrado de datos sensibles mediante AES-256, el índice ciego y la capa de abstracción para servicios de notificación. Los riesgos R001 y

R04 requieren acciones continuas de documentación y monitoreo de dependencias. El riesgo R002, aunque de baja probabilidad actualmente, debe ser monitoreado a medida que el sistema crezca en producción.

17. Accesibilidad y Experiencia de Usuario

17.1. Cumplimiento de accesibilidad

El sistema ha sido evaluado en cumplimiento de los estándares WCAG 2.1 (niveles A y AA), Sección 508 y EN 301 549, utilizando herramientas como axe DevTools y Lighthouse.

Cuadro 30: Puntajes de accesibilidad por página y entorno

Página	Escritorio	Móvil
Página principal	98	93
Registro de ESAVI	89	89
Panel informativo	98	95

Cuadro 31: Incidencias de accesibilidad identificadas

Problema	Impacto	Solución
Botones sin nombre accesible	Lectores de pantalla anuncian "botón" sin contexto	Asignar atributos <code>aria-label</code>
Barras de progreso sin etiqueta	No se informa el propósito del indicador	Etiquetar elemento con <code>aria-label</code>
Jerarquía de encabezados incorrecta	Dificulta navegación con lectores de pantalla	Revisar estructura semántica H1-H6

17.2. Principios de UX aplicados

El diseño de la interfaz se fundamenta en las heurísticas de usabilidad de Nielsen [6].

Cuadro 32: Heurísticas de Nielsen aplicadas al sistema

Heurística	Aplicación en el sistema
Visibilidad del estado del sistema	Indicadores de progreso, confirmaciones y notificaciones de estado.
Coincidencia mundo real-sistema	Terminología médica estándar, lenguaje claro y cercano.
Control y libertad del usuario	Navegación entre pasos, cancelación de acciones y retroceso.
Consistencia y estándares	Patrones de interfaz uniformes y navegación consistente.
Prevención de errores	Validaciones en tiempo real y confirmación antes de acciones críticas.
Reconocimiento antes que recuerdo	Menús visibles, autocompletado y información contextual.
Flexibilidad y eficiencia	Atajos para usuarios frecuentes y flujos optimizados.
Diseño estético y minimalista	Interfaz limpia, sin elementos distractores.
Recuperación de errores	Mensajes claros con sugerencias de corrección.
Ayuda y documentación	Tooltips en campos complejos y sección de preguntas frecuentes.

17.3. Evaluación de usabilidad

La evaluación empleó un recorrido cognitivo con 7 participantes (5 ciudadanos, 2 administradores) y el cuestionario SUS. Cada participante realizó tareas guiadas sin instrucciones previas y respondió el cuestionario.

Cuadro 33: Resultados del cuestionario SUS por participante

Afirmación	P1	P2	P3	P4	P5	A1	A2
1. Usaría el sistema frecuentemente	4	5	4	4	5	4	5
2. Sistema innecesariamente complejo	1	1	2	1	1	2	1
3. Sistema fácil de usar	5	4	4	5	4	4	4
4. Necesitaría ayuda técnica	1	2	1	1	2	1	2
5. Funciones bien integradas	4	5	4	4	5	4	4
6. Demasiada inconsistencia	1	1	2	1	1	2	1
7. Fácil de aprender	5	5	4	5	5	5	5
8. Sistema incómodo de usar	1	1	1	1	1	1	1
9. Me sentí seguro al usarlo	4	5	4	5	4	4	4
10. Necesité aprender antes	1	1	2	1	1	1	1
Puntaje SUS	92.5	95.0	80.0	95.0	92.5	85.0	90.0

Cuadro 34: Resumen de resultados SUS

Métrica	Valor
Puntaje SUS promedio	90.0/100
Categoría (Bangor et al.)	Excelente
Rango de puntajes	80.0 – 95.0
Mejor valorada	Facilidad de aprendizaje (4.9/5)

Los resultados son representativos solo para ciudadanos y administradores. Quedan pendientes pruebas con médicos y jefes municipales.

17.4. Requisitos previos a la puesta en producción

1. Completar pruebas de usabilidad con médicos y jefes municipales (mínimo 3 por perfil).
2. Corregir incidencias de accesibilidad identificadas en el Registro de ESAVI.
3. Realizar segunda iteración con rediseño según hallazgos.

18. Limitaciones y Trabajo Futuro

18.1. Limitaciones conocidas

Cuadro 35: Limitaciones del sistema

ID	Limitación	Categoría	Impacto
L001	Sin soporte para modo offline	Técnica	Alto
L002	Servicios de notificación en prototipo (EmailJS, OpenWA)	Técnica	Alto
L003	Pruebas de usabilidad sin médicos ni jefes municipales	Validación	Alto
L004	Desplegado en Railway/Vercel, no en servidores IFV	Infraestructura	Alto
L005	Duplicados descartan copia sin fusionar datos	Funcional	Bajo
L006	Máximo 70 usuarios concurrentes	Arquitectura	Medio
L007	Solo disponible en idioma español	Negocio	Medio

18.2. Roadmap de mejoras

Cuadro 36: Backlog de mejoras

ID	Mejora	Prioridad	Versión estimada
M001	Migrar notificaciones a SMTP institucional y WhatsApp Business	Alta	v1.0
M002	Completar pruebas de usabilidad con médicos y jefes municipales	Alta	v1.0
M003	Migrar a servidores físicos del Instituto Finlay	Alta	v1.0
M004	Optimizar recursos estáticos del frontend (WebP, precarga, JS)	Media	v1.1
M005	Implementar fusión inteligente de duplicados	Media	v1.1
M006	App móvil nativa (iOS/Android)	Media	v2.0
M007	Reportes personalizables	Baja	v1.5
M008	Integración con SAP	Baja	v2.1

A. Manual de Instalación para Evaluadores

A.1. Requisitos previos

- **Docker** y **Docker Compose** (para ejecutar los contenedores definidos en `esavi-report/backend/docker`)
- **.NET SDK 8** (versión utilizada en el proyecto; se recomienda la misma para evitar incompatibilidades).
- **dotnet-ef** (herramienta requerida para ejecutar migraciones):

```
dotnet tool install --global dotnet-ef
```

- **Node.js 18+** y **npm** (para el frontend). El frontend usa Vite y dispone de los scripts `dev`, `build` y `preview` en `package.json`.

A.2. Pasos para el entorno de desarrollo con contenedores

1. Abrir una terminal en `esavi-report/backend` y levantar los servicios con Docker Compose:

```
cd esavi-report/backend
docker compose up -d --build
```

2. Ejecutar las migraciones. El repositorio incluye el script `start_migrations.sh` que aplica las migraciones a la base de datos y arranca la API:

```
./start_migrations.sh
```

Nota: el script asume que `dotnet-ef` está disponible. Si no lo tiene, instálelo con el comando indicado en la sección de requisitos previos. Alternativamente, puede ejecutar las migraciones manualmente:

```
dotnet ef database update --project ./Finlay.PharmaVigilance.
Infrastructure/
```

3. Levantar el frontend. Para desarrollo local:

```
cd esavi-report/frontend
npm install
npm run dev
```

El frontend estará disponible en `http://localhost:5173`.

A.3. Verificaciones después del arranque

- Listar contenedores y comprobar estados:

```
docker ps
```

Busque los contenedores: `mysql`, `rabbitmq`, `api` (y opcionalmente `frontend`).

- Ver la interfaz de administración de RabbitMQ:

```
http://localhost:15672 (usuario: guest, contraseña: guest)
```

- Abrir Swagger de la API para probar endpoints. La API expone Kestrel en el puerto 5137:

```
http://localhost:5137/swagger
```

- Consultar logs de la API si algo falla:

```
docker compose logs api --tail 200
```

- Conectarse a la base de datos MySQL. Las credenciales se definen en el archivo `docker-compose.yml` del backend:

```
mysql -h localhost -P 3306 -u root -p
```

A.4. Ejecución de la API sin Docker

Alternativamente, se puede ejecutar la API directamente sin contenedores:

```
cd esavi-report/backend
dotnet build
dotnet run --project ./Finlay.PharmaVigilance.API/
```

Si ejecuta la API localmente, Swagger estará disponible en <http://localhost:5137/swagger>.

A.5. Solución de problemas comunes

- **Las migraciones fallan:** verifique que `dotnet-ef` esté instalado y que las cadenas de conexión apunten al contenedor o servidor correcto.
- **MySQL no inicia o no acepta conexiones:** verifique `docker compose logs mysql` y los volúmenes persistentes (permisos o espacio en disco).
- **La API no responde en el puerto 5137:** compruebe `docker compose logs api` y que la variable `ASPNETCORE_ENVIRONMENT` esté establecida (`docker-compose configura Development`).
- **El frontend con Vite muestra errores al iniciar:** ejecute `npm install` para asegurarse de que las dependencias estén instaladas, y revise la salida de `npm run dev`.
- **Error de conexión a RabbitMQ:** verifique que el contenedor `rabbitmq` esté corriendo y que los puertos 5672 y 15672 estén disponibles.

B. Glosario de Términos Técnicos

- **JWT:** JSON Web Token. Estándar abierto (RFC 7519) para transmitir información verificable entre partes mediante un token firmado digitalmente. Utilizado en el sistema para autenticación y control de acceso basado en roles.
- **ORM:** Object-Relational Mapping. Patrón de diseño que permite mapear entidades del dominio a tablas de una base de datos relacional. En el sistema, implementado mediante Entity Framework Core.
- **CI/CD:** Continuous Integration / Continuous Deployment. Conjunto de prácticas para automatizar la compilación, pruebas y despliegue de aplicaciones.
- **MassTransit:** Librería .NET para construir buses de mensajería. En el sistema, se utiliza para la integración con RabbitMQ en la capa de infraestructura.
- **RabbitMQ:** Broker de mensajería que implementa el protocolo AMQP. Utilizado en el sistema para la comunicación asíncrona entre componentes.
- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones multi-contenedor. En el sistema, se utiliza para orquestar los servicios de backend, base de datos y mensajería.

- **Vite:** Herramienta de desarrollo y empaquetamiento para aplicaciones web. Utilizada en el frontend para el entorno de desarrollo y la generación del build de producción.
- **Serilog:** Biblioteca de logging estructurado para .NET. Utilizada en el backend para generar salidas de log a consola y archivos, con soporte para múltiples sinks.
- **Entity Framework Core:** ORM de Microsoft para .NET. Utilizado en el sistema para el acceso a datos, gestión de migraciones y mapeo objeto-relacional.
- **Swagger:** Herramienta para documentar y probar APIs REST. En el sistema, expone los endpoints disponibles en `/swagger` para facilitar las pruebas durante el desarrollo.
- **Blind Index:** Índice ciego. Técnica que almacena un hash de un dato sensible (como el número de identidad) para permitir búsquedas exactas sin exponer el dato original. Utilizado en el sistema para detectar reportes duplicados sin comprometer la privacidad.
- **CAPTCHA:** Prueba de Turing para distinguir humanos de bots. En el sistema se implementa mediante Friendly Captcha, un servicio basado en prueba de trabajo (proof-of-work) que protege las rutas públicas sin interrumpir al usuario.

C. Referencias Bibliográficas y Normativas

Referencias

- [1] Kaliski, B. (2000). PKCS #5: Password-Based Cryptography Specification Version 2.0. IETF, RFC 2898. <https://datatracker.ietf.org/doc/html/rfc2898>
- [2] Microsoft. (2024). PasswordHasher;TUser;Class. <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.passwordhasher-1>
- [3] Oracle Corporation. (2024). MySQL 8.0 Reference Manual: Transaction Isolation Levels. <https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html>
- [4] Friendly Captcha GmbH. (2024). Friendly CAPTCHA Documentation. <https://docs.friendlycaptcha.com/>
- [5] Microsoft. (2024). Rate limiting middleware in ASP.NET Core. <https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit>
- [6] Nielsen, J. (1994). Heuristic Evaluation. In Nielsen, J. & Mack, R.L. (Eds.), Usability Inspection Methods (pp. 25-62). John Wiley & Sons.